PYTHON PROGRAMMING

CASE STUDY ASSIGNMENT – 10

Case Study 10 – Bus Passenger Occupancy Simulation

1. Problem Statement

A bus transportation system needs to track the number of passengers boarding and leaving a bus during its journey. Develop a Python program to simulate passenger occupancy by accepting the number of passengers boarding and getting off at each stop. The program should update the current passenger count, ensure that the passenger count never becomes negative or exceeds the bus's maximum capacity, and display the number of passengers onboard after each stop.

2. Objectives

- **To simulate passenger boarding and exit at each bus stop.**
- **To maintain the current number of passengers onboard.**
- **To ensure the passenger count does not exceed the bus's maximum capacity.**
- **To prevent the passenger count from becoming negative.**
- **To display the passenger count after each stop.**
- **To calculate the total number of passengers who boarded and exited during the journey.**
- **To apply Python concepts such as variables, loops, conditional statements, and arithmetic operations.**

3. Assumptions

1. **The bus has a fixed maximum capacity of 50 passengers.**
2. **The bus starts its journey with zero passengers.**
3. **The number of passengers boarding and exiting is entered by the user at each stop.**
4. **The number of boarding and exiting passengers must be non-negative integers.**
5. **Passengers cannot exit if there are not enough passengers onboard.**

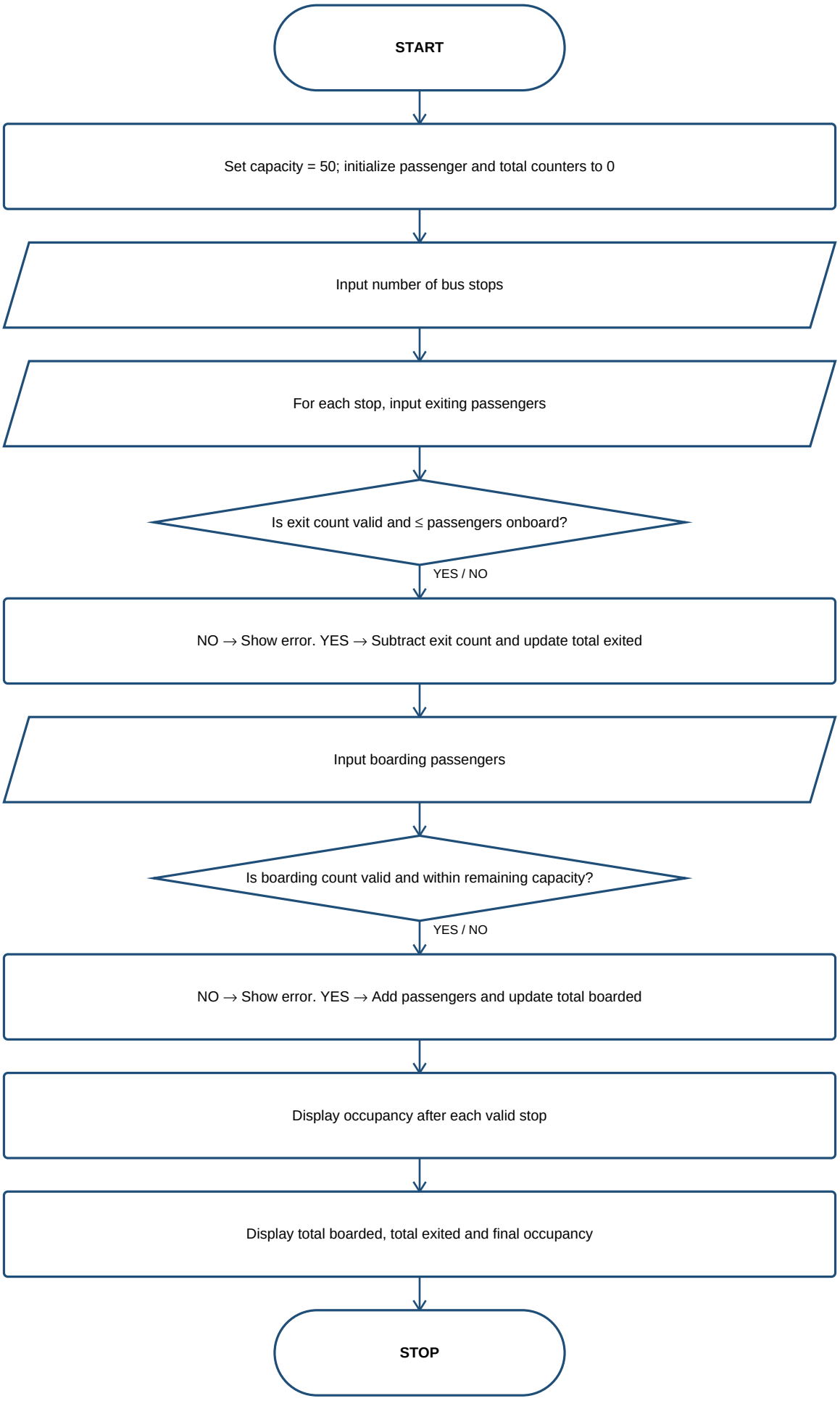
6. **The total number of passengers onboard must never exceed the bus capacity.**
7. **The program processes a fixed number of bus stops.**
8. **Passenger occupancy is updated only when the boarding and exit numbers are valid.**
9. **The program displays the passenger count after every valid stop.**

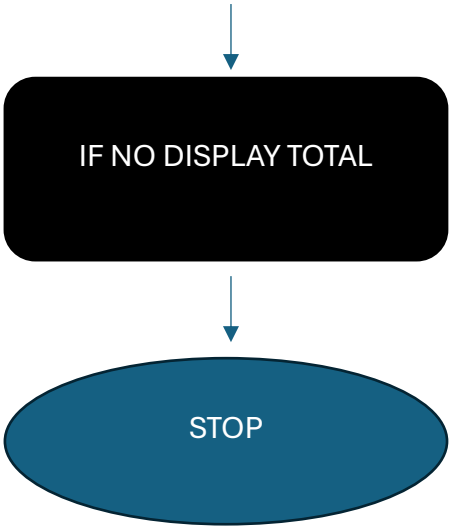
4. Algorithm

1. **Start.**
2. **Set the maximum bus capacity to 50 passengers.**
3. **Initialize the current passenger count, total boarded passengers, and total exited passengers to zero.**
4. **Input the number of bus stops.**
5. **Repeat the following steps for each stop:**
    - **Input the number of passengers getting off the bus.**
    - **Check whether the exit count is valid and does not exceed the current passenger count.**
    - **If invalid, display an error message.**
    - **Otherwise, subtract the exiting passengers from the current passenger count.**
    - **Input the number of passengers boarding the bus.**
    - **Check whether the boarding count is valid and does not exceed the remaining capacity.**
    - **If invalid, display an error message.**
    - **Otherwise, add the boarding passengers to the current passenger count.**
    - **Update the total number of passengers boarded and exited.**
    - **Display the current passenger count.**
6. **Display the total passengers boarded, total passengers exited, and final occupancy.**
7. **Stop.**

5. FLOWCHART

Bus Passenger Occupancy Simulation





6. Pseudocode

```
BEGIN
capacity = 50
   passengers = 0
   total_boarded = 0
   total_exited = 0

   INPUT number_of_stops

   FOR each stop from 1 to number_of_stops

      INPUT exiting_passengers

      IF exiting_passengers < 0 OR
        exiting_passengers > passengers THEN
DISPLAY "Invalid number of exiting passengers"
```

```
    ELSE

        passengers = passengers - exiting_passengers
        total_exited = total_exited + exiting_passengers

        INPUT boarding_passengers

        IF boarding_passengers < 0 OR
          passengers + boarding_passengers > capacity THEN

            DISPLAY "Invalid boarding count or bus capacity exceeded"

        ELSE

            passengers = passengers + boarding_passengers
            total_boarded = total_boarded + boarding_passengers

            DISPLAY "Passengers onboard:", passengers

        END IF

    END IF

END FOR
```

DISPLAY "Total passengers boarded:", total_boarded

DISPLAY "Total passengers exited:", total_exited

DISPLAY "Final occupancy:", passengers

END

7.Source code

Bus Passenger Occupancy Simulation.py

PYTHON-ACTIVITY-BASED-LEARNING > Bus Passenger Occupancy Simulation.py > ...

```python

capacity = 50
passengers = 0
total_boarded = 0
total_exited = 0

number_of_stops = int(input("Enter number of bus stops: "))

for stop in range(1, number_of_stops + 1):
    print("\nBus Stop", stop)

    exiting = int(input("Enter passengers getting off: "))

    if exiting < 0 or exiting > passengers:
        print("Invalid number of exiting passengers")
        continue

    passengers -= exiting
    total_exited += exiting

    boarding = int(input("Enter passengers boarding: "))

    if boarding < 0 or passengers + boarding > capacity:
        print("Invalid boarding count or bus capacity exceeded")
        continue

    passengers += boarding
    total_boarded += boarding

    print("Passengers onboard:", passengers)

print("\nJourney Summary")
print("Total passengers boarded:", total_boarded)
print("Total passengers exited:", total_exited)
print("Final occupancy:", passengers)
print("Available seats:", capacity - passengers)
```

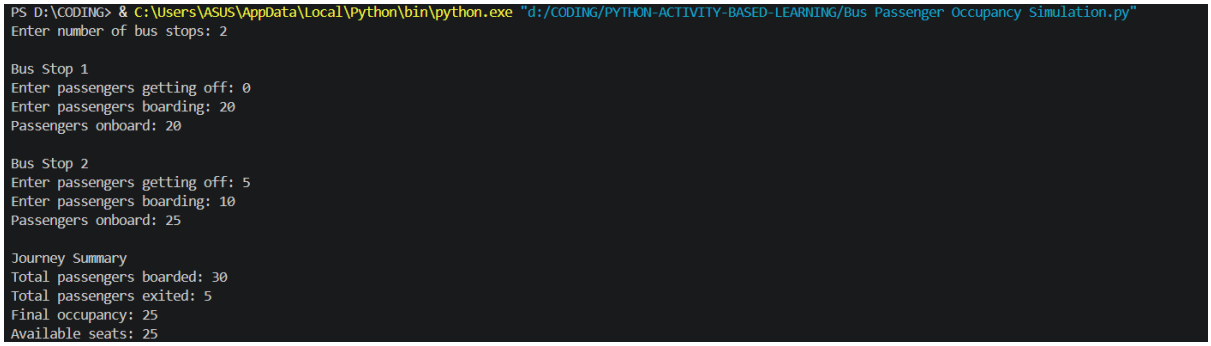
8. Test Cases

Test Case	Bus Stop	Passengers Exiting	Passengers Boarding	Expected Output
1	1	0	20	Passengers onboard: 20
2	2	5	10	Passengers onboard: 25

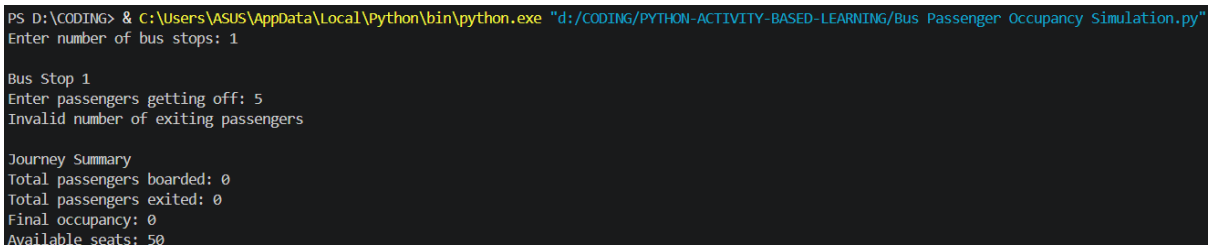
3	1	30	—	Invalid number of exiting passengers (if fewer than 30 passengers are onboard)
4	1	0	55	Invalid boarding count or bus capacity exceeded
5	1	-2	—	Invalid number of exiting passengers
6	1	0	0	Passengers onboard: 0

9. Output Screenshots

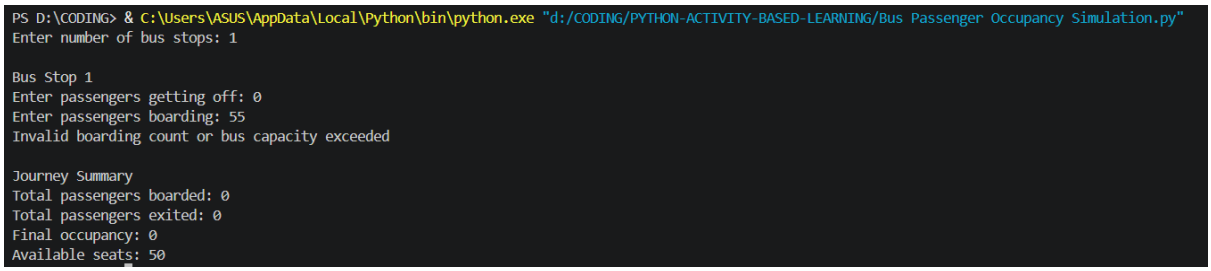
Screenshot 1: Successful Passenger Journey



Screenshot 2: Invalid Exit Count



Screenshot 3: Bus Capacity Exceeded



10. Conclusion

The Bus Passenger Occupancy Simulation was successfully developed using Python. The program tracks passengers boarding and exiting at each bus stop, maintains the current passenger count, and ensures that the bus capacity of 50 passengers is not exceeded. It also validates passenger exit counts and displays the total passengers boarded, total passengers exited, final occupancy, and available seats. This project demonstrates the practical use of variables, loops, conditional statements, arithmetic operations, and input validation in Python.